

Day 81: Basic Train/Test Data Preparation

1. Day Number

Day 81

2. Topic Name

Preparing Features and Labels for Machine Learning

Today you will learn how to organize a dataset into:

- **Features** — the information given to a model
- **Label/target** — the value the model will eventually try to predict
- **Training data** — data used for learning
- **Test data** — data kept aside for checking performance later

You will **not train a machine-learning model yet**.

3. Connection

Yesterday, you practiced **Exploratory Data Analysis (EDA)**.

You learned how to:

- inspect a dataset
- check columns and data types
- find missing values
- calculate summaries
- visualize patterns

Now that the dataset is understood and reasonably clean, the next question is:

| How do we organize this data before giving it to a machine-learning model?

That is today's goal.

4. Important Topics

Feature columns

Features are the input values used to make a prediction.

For example, suppose we have:

	Size	Price
	800	40
	1000	50
	1200	60

	Size	Price
	1500	75
	1800	90

If we eventually want to predict house price, then:

Size = feature

Label / Target column

The target is the value we want the model to predict.

In our example:

Price = target

You may see both words:

- label
- target

For beginner ML problems, they usually mean the value we are trying to predict.

X

By convention, machine-learning code often stores the features in a variable called:

X

So conceptually:

X = Size column

y

The target is commonly stored in:

y

Conceptually:

y = Price column

A useful memory trick is:

X → questions / inputs
y → answer / target

Training data

Training data is the part of the dataset that a machine-learning model will eventually learn from.

For example:

```
House 1  
House 2  
House 3  
House 4
```

could become training data.

Test data

Test data is kept separate.

For example:

```
House 5
```

might be kept as test data.

Later, after a model has learned from the training data, we can test it using data it did not train on.

5. Foundational Notes

Imagine you are studying for an exam.

If someone gives you:

```
Question 1 + answer  
Question 2 + answer  
Question 3 + answer
```

you can study them.

Those are like **training data**.

But suppose the final exam contains exactly the same questions.

Getting them correct would not prove that you actually learned the subject. You might simply remember the answers.

Machine learning has a similar problem.

Therefore, we usually divide our data into:

```
Original Dataset
|
+----- Training Data
|
+----- Test Data
```

The training set is used for learning.

The test set is reserved for evaluation.

6. Why Shouldn't We Train and Evaluate on Exactly the Same Data?

Suppose a dataset contains 100 houses.

If we train a model using all 100 houses and then test the model using those exact same 100 houses, the test is not very meaningful.

The model has already seen them.

We want to know something more useful:

| Can the model handle data it did not see during training?

So we might conceptually use:

```
80 houses → training
20 houses → testing
```

This is called a **train/test split**.

A common beginner example is:

```
80% training
20% testing
```

But this is not a universal rule. Different projects may use different proportions.

7. Easy Example

Consider this small house dataset:

```
house_data = {
    "size": [800, 1000, 1200, 1500, 1800],
    "price": [40, 50, 60, 75, 90]
}
```

Think of the values as:

Size	Price
800	40
1000	50
1200	60
1500	75
1800	90

Suppose the future goal is:

| Predict house price from house size.

Then:

Feature:
size

Target:
price

So conceptually:

X = size

y = price

You could then keep some rows for training and some rows for testing.

For example:

Training:
800 → 40
1000 → 50
1200 → 60
1500 → 75

Testing:
1800 → 90

This is only illustrating the idea. With real machine-learning work, you would normally split the rows using a library rather than manually choosing them.

8. Problem Statement

Create a tiny dataset containing house information.

Use two columns:

size

price

Example idea:

Size	Price
700	35
900	45
1100	55
1400	70
1700	85

Your task is to:

1. Create the small house dataset.
2. Identify `size` as the **input feature**.
3. Identify `price` as the **target**.
4. Store the feature data conceptually as `X`.
5. Store the target data conceptually as `y`.
6. Divide the rows into a small **training set** and **test set**.
7. Inspect the shapes of the resulting data.
8. Do **not** train a machine-learning model.

9. Concepts Used

You will practice concepts you already know together with a few new ML ideas:

- Pandas DataFrame
- rows and columns
- selecting DataFrame columns
- feature
- target
- `X`
- `y`
- training set
- test set
- dataset shape
- separating input from output

The basic flow is:

```
Dataset
  ↓
Choose feature
  ↓
```

```
Choose target
↓
X and y
↓
Train/Test Split
↓
X_train
X_test
y_train
y_test
```

10. Thought Process

When preparing data for machine learning, ask these questions in order.

Question 1: What am I trying to predict?

For this problem:

```
house price
```

Therefore:

```
price = target
```

Question 2: What information will help predict it?

We have:

```
house size
```

Therefore:

```
size = feature
```

Question 3: Which data goes into **x**?

The feature column:

```
X → size
```

Question 4: Which data goes into **y**?

The target column:

```
y → price
```

Question 5: Should every row be used for training?

No.

Keep some rows aside for testing.

Question 6: Do the feature and target still match?

This is extremely important.

If this row says:

```
size = 1200  
price = 60
```

those two values belong together.

When splitting data, we must preserve that relationship.

11. Beginner-Friendly Pseudocode

```
START  
  
import the data library  
  
create a small house DataFrame  
    size  
    price  
  
inspect the DataFrame  
  
select the size column as X  
  
select the price column as y  
  
check the shape of X  
check the shape of y  
  
split X and y into:  
    training features  
    test features  
    training targets  
    test targets  
  
print or inspect their shapes  
  
STOP
```

Conceptually, the result should look like:

```
X
↓
size values

y
↓
price values
```

Then:

```
X_train    → features used for training
y_train    → correct targets for training

X_test     → features saved for testing
y_test     → correct targets saved for testing
```

12. Suggested Solving Approach

Use a **simple data-preparation approach**.

First, create a tiny Pandas DataFrame.

Then identify your problem:

```
Input: house size
Output: house price
```

Next, separate the dataset:

```
DataFrame
|
+---- X → size
|
+---- y → price
```

Then conceptually divide them:

```
X → X_train + X_test
y → y_train + y_test
```

Later, when you begin using machine-learning libraries, you will commonly encounter a utility such as:

```
train_test_split(...)
```

For today's exercise, focus on understanding **what the split means**, rather than training anything.

13. Easy Edge Cases

Extremely small dataset

Imagine you only have:

2 houses

A train/test split becomes difficult because there is barely enough information for either training or testing.

For learning exercises, a tiny dataset is fine.

For real machine learning:

more useful data is usually needed

Missing target

Suppose you have:

Size	Price
800	40
1000	?
1200	60

The second row has no correct target value.

That creates a problem because we cannot properly use that example for normal supervised training unless we decide how to handle the missing target.

This connects directly to your earlier lessons on **missing-value cleaning**.

14. Expected Data Shapes

Suppose your dataset contains:

5 houses

and one feature:

size

If you keep ☒ as a DataFrame with one feature column, its conceptual shape is:

(5, 1)

This means:

5 rows
1 feature column

Your target might have:

5 values

so y may appear conceptually as:

(5,)

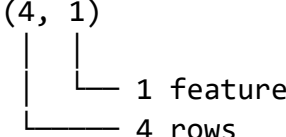
If you use four rows for training and one for testing, you might see shapes similar to:

```
X_train → (4, 1)
X_test  → (1, 1)

y_train → (4,)
y_test  → (1,)
```

Do not worry if this notation feels unfamiliar.

Remember:

(4, 1)

1 feature
4 rows

15. Hint Only

Start by asking:

What am I predicting?

Answer:

price

So that should become your y .

Then ask:

What information am I using to make the prediction?

Answer:

size

So that should become your `X`.

After creating `X` and `y`, think about how a train/test splitting utility could divide them **together** so that every house size remains paired with its correct price.

Your goal today is only:

Raw Data

↓

Features + Target

↓

Training Data + Test Data

Do not fit or predict with a machine-learning model yet.